D-ANMOL IT SOLUTIONS

Python Programming

Learn Python from fundamentals to practical application development.

3 Months | Foundation Track

A step-by-step practical learning roadmap with explained tools, applied examples, project milestones and career preparation.

01 WHAT CAN YOU BUILD AFTER THIS COURSE?

- Student Record Manager: Create a menu-driven application to add, search, edit and save student records.
- Expense Tracker Application: Store expenses, filter by category and produce monthly summaries.

Possible career directions

Python developer trainee, automation developer trainee and backend programming intern. Roles are illustrative and depend on skills, experience and hiring criteria.

Your learning journey

From learning to practical delivery

Learn concepts	**Practise skills**	**Build projects**	**Show portfolio**

02 STEP-BY-STEP COURSE SYLLABUS

Modules are ordered from foundations to application. Each includes what you learn, why it matters and a practical classroom activity.

STEP 01 | Python Setup & First Programs

What you will study:

- Python installation: understand the concept, practise it with a guided example, and apply it in a relevant task.
- IDE: understand the concept, practise it with a guided example, and apply it in a relevant task.
- interpreter: understand the concept, practise it with a guided example, and apply it in a relevant task.
- syntax: understand the concept, practise it with a guided example, and apply it in a relevant task.
- comments: understand the concept, practise it with a guided example, and apply it in a relevant task.
- print and input: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: It provides a foundation for later modules and real project implementation.

Hands-on example: Create and run basic Python scripts.

STEP 02 | Variables, Data Types & Operators

What you will study:

• Numbers: understand the concept, practise it with a guided example, and apply it in a relevant task.

• strings: understand the concept, practise it with a guided example, and apply it in a relevant task.

• booleans: understand the concept, practise it with a guided example, and apply it in a relevant task.

• casting: understand the concept, practise it with a guided example, and apply it in a relevant task.

• arithmetic: understand the concept, practise it with a guided example, and apply it in a relevant task.

• comparison and logical operators: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: It provides a foundation for later modules and real project implementation.

Hands-on example: Build a simple calculator. D-ANMOL IT SOLUTIONS | COURSE CURRICULUM Page 2

STEP 03 | Conditions & Loops

What you will study:

• if/elif/else: understand the concept, practise it with a guided example, and apply it in a relevant task.

• for: understand the concept, practise it with a guided example, and apply it in a relevant task.

• while: understand the concept, practise it with a guided example, and apply it in a relevant task.

• range: understand the concept, practise it with a guided example, and apply it in a relevant task.

• break: understand the concept, practise it with a guided example, and apply it in a relevant task.

• continue and nested logic: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: It provides a foundation for later modules and real project implementation.

Hands-on example: Create menu-driven programs and pattern exercises.

STEP 04 | Strings & Collections

What you will study:

• String methods: understand the concept, practise it with a guided example, and apply it in a relevant task.

• lists: understand the concept, practise it with a guided example, and apply it in a relevant task.

• tuples: understand the concept, practise it with a guided example, and apply it in a relevant task.

• sets: understand the concept, practise it with a guided example, and apply it in a relevant task.

• dictionaries: understand the concept, practise it with a guided example, and apply it in a relevant task.

• indexing and iteration: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: You use this skill to solve practical problems, improve accuracy and build reliable outputs.

Hands-on example: Build a contact or inventory manager.

STEP 05 | Functions & Modules

What you will study:

• Parameters: understand the concept, practise it with a guided example, and apply it in a relevant task.

• return values: understand the concept, practise it with a guided example, and apply it in a relevant task.

• scope: understand the concept, practise it with a guided example, and apply it in a relevant task.

• lambda basics: understand the concept, practise it with a guided example, and apply it in a relevant task.

• imports and reusable code: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: You use this skill to solve practical problems, improve accuracy and build reliable outputs.

Hands-on example: Refactor a script into reusable functions.

STEP 06 | Files, Exceptions & JSON

What you will study:

• Read/write files: understand the concept, practise it with a guided example, and apply it in a relevant task.

• CSV: understand the concept, practise it with a guided example, and apply it in a relevant task.

• JSON: understand the concept, practise it with a guided example, and apply it in a relevant task.

• try/except and context managers: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: You use this skill to solve practical problems, improve accuracy and build reliable outputs.

Hands-on example: Create a file-based record management utility.

STEP 07 | Object-Oriented Programming

What you will study:

• Classes: understand the concept, practise it with a guided example, and apply it in a relevant task.

• objects: understand the concept, practise it with a guided example, and apply it in a relevant task.

• constructors: understand the concept, practise it with a guided example, and apply it in a relevant task.

• inheritance: understand the concept, practise it with a guided example, and apply it in a relevant task.

• encapsulation and polymorphism: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: You use this skill to solve practical problems, improve accuracy and build reliable outputs.

Hands-on example: Model a simple library or student system.

STEP 08 | Working with Packages & APIs

What you will study:

• pip: understand the concept, practise it with a guided example, and apply it in a relevant task.

• virtual environments: understand the concept, practise it with a guided example, and apply it in a relevant task.

• requests: understand the concept, practise it with a guided example, and apply it in a relevant task.

• HTTP basics and API response handling: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: You use this skill to solve practical problems, improve accuracy and build reliable outputs.

Hands-on example: Consume a public API and process JSON data.

STEP 09 | Databases with Python

What you will study:

• SQL basics: understand the concept, practise it with a guided example, and apply it in a relevant task.

• SQLite/MySQL concepts: understand the concept, practise it with a guided example, and apply it in a relevant task.

• CRUD operations and parameterised queries: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: You use this skill to solve practical problems, improve accuracy and build reliable outputs.

Hands-on example: Build a database-backed mini application.

STEP 10 | Testing, Debugging & Git

What you will study:

• Debugging: understand the concept, practise it with a guided example, and apply it in a relevant task.

• assertions: understand the concept, practise it with a guided example, and apply it in a relevant task.

• unit testing basics: understand the concept, practise it with a guided example, and apply it in a relevant task.

• Git commits and project structure: understand the concept, practise it with a guided example, and apply it in a relevant task.

Why this matters: You use this skill to solve practical problems, improve accuracy and build reliable outputs.

Hands-on example: Test and version-control a Python project.

03 TOOLS, LIBRARIES & THEIR USES

This section explains where each tool or library fits in a practical project. Some advanced tools may be introduced as demonstrations in shorter tracks.

Tool / library	What it is used for
Python standard library	Handle files, dates, JSON and common tasks without extra packages.
Requests	Send HTTP requests and read API responses.
SQLite	Persist structured data in a local database.
Matplotlib	Plot numerical results and summaries.
pytest / unittest	Verify that code behaves as expected.

04 PROJECT-BASED LEARNING

Included: 2 project(s). Every project includes planning, implementation, testing and a presentation of results.

NORMAL PROJECT 1: Student Record Manager

What you will build: Create a menu-driven application to add, search, edit and save student records.

Where tools are used: Python functions, dictionaries, file handling and JSON.

Final deliverable: Runnable application and sample dataset.

Project workflow

Input / brief	Build & test	Review results	Present output

Live project means an applied, end-to-end project based on a realistic business scenario or suitable dataset; access to an external client or production deployment is not implied.

LIVE PROJECT 2: Expense Tracker Application

What you will build: Store expenses, filter by category and produce monthly summaries.

Where tools are used: Python classes, CSV/SQLite and Matplotlib for summaries.

Final deliverable: CLI app, data storage and report.

Project workflow

Input / brief	Build & test	Review results	Present output

Live project means an applied, end-to-end project based on a realistic business scenario or suitable dataset; access to an external client or production deployment is not implied.

05 TRAINING PLAN & ASSESSMENTS

Suggested duration: 3 Months (approximately 12 weeks). Exact session hours and timetable should be confirmed with the institute.

- Month 1 — Foundations and tool practice
- Month 2 — Intermediate topics, assignments and normal project
- Month 3 — Live project and final review

Evaluation: practical exercises, module checkpoints, code/document review where applicable, project demonstration and final feedback.

FREE SUPPORT INCLUDED WITH EVERY COURSE

1 YEAR DOUBT SUPPORT + INTERVIEW PREPARATION — FREE OF COST

- One year of doubt-clearing support related to the enrolled course.

- Interview preparation support at no additional course fee.
- Guidance on explaining course projects and strengthening fundamental concepts.
- Availability, format and scheduling of support sessions to be communicated by the institute.

Important: This brochure presents the requested training offer. The institute should confirm the exact support policy, availability, start date and any applicable conditions before publishing. Interview preparation is not a job or placement guarantee.

D-Anmol IT Solutions | Course information & admissions: https://danmolitsolutions.vercel.app/